Manejo de Archivos y Persistencia de Datos

Hasta ahora, todos nuestros programas han tenido una limitación fundamental: en cuanto se cierran, olvidan todo. Los datos que calculamos, las listas que creamos, todo desaparece.

Hoy vamos a romper esa barrera. Aprenderemos a darle "memoria" a nuestros programas, permitiéndoles leer y escribir información en archivos. Esta habilidad es la puerta de entrada para trabajar con datos del mundo real, desde simples archivos de configuración hasta grandes datasets.

Objetivos de la Clase:

Al finalizar esta sesión, serás capaz de:

- Justificar la necesidad de la persistencia de datos para que los programas sean útiles.
- Leer y escribir archivos de texto plano (`.txt`) de forma segura utilizando el gestor de contexto de Python.
- Comprender los diferentes modos de apertura de un archivo (`'r'`, `'w'`, `'a'`).
- Interactuar con el sistema operativo. Rutas relativas y absolutas.
- Procesar datos tabulares (filas y columnas) utilizando el módulo `csv`, la herramienta estándar de Python para esta tarea.
- Construir un programa completo que siga el ciclo de vida de los datos: **lectura -> procesamiento -> escritura**.

---

1. El Problema: La Memoria Volátil de Nuestros Programas

Imaginemos que construimos nuestro "Gestor de Contactos" del ejercicio de la clase pasada. Agregamos 10 contactos, cerramos la terminal y la volvemos a abrir. ¿Qué pasó? Perdimos todo.

Esto ocurre porque los datos almacenados en variables viven en la memoria RAM, que es **volátil**. Al terminar el programa, esa memoria se libera. Para que la información sobreviva, necesitamos guardarla en un medio de almacenamiento **persistente**, como el disco duro. La forma más directa de hacerlo es a través de archivos.

---

2. La Solución Fundamental: Archivos de Texto Plano (`.txt`)

Un archivo de texto plano es el tipo de archivo más simple que existe. Es una secuencia de líneas de texto, ideal para guardar logs, notas, o reportes simples. El proceso siempre sigue tres pasos conceptuales: ABRIR, OPERAR (leer/escribir) y CERRAR.

Buena Práctica: El Gestor de Contexto `with open(...)`

Podríamos abrir y cerrar archivos manualmente, pero es arriesgado. Si ocurre un error mientras el archivo está abierto, podría no cerrarse correctamente y corromperse. Python nos da una sintaxis mucho más segura y elegante:

```
with open('nombre_del_archivo.txt', 'modo') as variable_archivo:

    # Todo el código que opera con el archivo va aquí, indentado.
    # Python se encarga de cerrarlo automáticamente al salir de este
bloque.
```

Los **modos** más comunes son:

- `'r'`: **Read** (Leer). El archivo debe existir, o tendremos un error.
- `'w'`: **Write** (Escribir). Si el archivo no existe, lo crea. Si existe, **borra todo su contenido** y escribe desde cero. ¡Usar con cuidado!
- `'a'`: **Append** (Añadir). Si el archivo no existe, lo crea. Si existe, agrega el nuevo contenido al final.

A. Escribir en un archivo (`.txt`)

```
# Lista de tareas pendientes
tareas_importantes = [
    "Preparar el material de la Clase 9.",
    "Corregir el primer parcial.",
    "Planificar el Trabajo Práctico."
]

# Usamos 'w' para crear (o sobrescribir) nuestro archivo de notas
with open('notas_profesor.txt', 'w') as f:
    f.write("--- Lista de Tareas ---\n")
    for tarea in tareas_importantes:
        f.write(f"- {tarea}\n")

print("Archivo 'notas_profesor.txt' generado con éxito.")
```

B. Leer un archivo (`.txt`)

```
# Usamos 'r' para leer el archivo que acabamos de crear
print("\n--- Leyendo el archivo de notas ---")
try:
    with open('notas_profesor.txt', 'r') as f:
        for linea in f:
            # .strip() es crucial para eliminar los saltos de línea
invisibles (\n)
            print(f"Línea leída: {linea.strip()}")
except FileNotFoundError:
    print("Error: El archivo no se encontró. ¿Seguro que lo creaste
primero?")
```

---

3. ¿Dónde se Guardan los Archivos? Rutas Absolutas y Relativas 📂

Una pregunta surge inmediatamente al ejecutar el código anterior: si solo escribimos `'notas_profesor.txt'`, ¿cómo sabe Python en qué carpeta crearlo?

La respuesta está en el Directorio de Trabajo Actual (*Current Working Directory* o CWD). Este es el punto de partida de nuestro script, la carpeta "donde se encuentra" al momento de la ejecución.

El Módulo `os`: Nuestro GPS para el Sistema de Archivos

Python tiene un módulo incorporado llamado `os` que nos permite interactuar con el sistema operativo. Su primera utilidad es, precisamente, preguntarle dónde estamos parados.

```
import os
directorio_actual = os.getcwd() # getcwd = Get Current Working Directory
print(f"Mi programa se está ejecutando en: {directorio_actual}")
```

Al usar una ruta simple como `'notas_profesor.txt'`, el archivo se creará dentro de ese directorio. Esto nos lleva a dos tipos de rutas:

1. Ruta Relativa: Es una dirección que se basa en nuestra ubicación actual (el CWD). Es como decir "la segunda puerta a la derecha".
    - `'notas.txt'` (el archivo está en la misma carpeta que mi script).
    - `'datos/ventas.csv'` (desde donde estoy, entró a la carpeta `datos` y busco `ventas.csv`).

2. Ruta Absoluta: Es la dirección completa y explícita desde la raíz del sistema de archivos. No depende de dónde se ejecute el script. Es como la dirección postal completa de una casa.
    - En Windows: `'C:\\Users\\MiUsuario\\Documentos\\notas.txt'`
    - En macOS/Linux: `'/home/miusuario/documentos/notas.txt'`

Las rutas relativas son geniales para proyectos autocontenidos, pero las absolutas son necesarias para acceder a archivos en ubicaciones fijas del sistema.

Buena Práctica: Construir Rutas de Forma Robusta con `os.path.join()`

¿Notaste que Windows usa `\` y macOS/Linux usan `/` como separador de carpetas? Si escribimos una ruta a mano (ej: `'datos/reporte.csv'`), nuestro código podría fallar en otro sistema operativo.

El módulo `os` nos da la solución perfecta y profesional: `os.path.join()`. Esta función construye una ruta de archivo usando el separador correcto para el sistema operativo donde se está ejecutando.

Ejemplo Práctico: Guardando un reporte en una subcarpeta

Imagina que queremos guardar nuestro reporte de tareas en una subcarpeta llamada `reportes`.

```
import os
# Lista de tareas (la misma de antes)
tareas_importantes = [
    "Preparar el material de la Clase 9.",
    "Corregir el primer parcial.",
    "Planificar el Trabajo Práctico."
]
# 1. Definimos el nombre de la carpeta y del archivo
carpeta_destino = 'reportes'
nombre_archivo = 'notas_profesor.txt'

# 2. Creamos la ruta completa de forma segura
ruta_completa = os.path.join(carpeta_destino, nombre_archivo)
print(f"El archivo se guardará en: {ruta_completa}") # -->
reportes/notas_profesor.txt o reportes\notas_profesor.txt

# 3. (Opcional pero recomendado) Nos aseguramos de que la carpeta exista
os.makedirs(carpeta_destino, exist_ok=True)
# 4. Usamos la ruta completa para escribir el archivo
with open(ruta_completa, 'w') as f:
    f.write("--- Reporte de Tareas Generado Automáticamente ---\n")
    for tarea in tareas_importantes:
        f.write(f"- {tarea}\n")
```

```
print(f"Archivo '{nombre_archivo}' generado con éxito en la carpeta
'{carpeta_destino}'.")
```

Al adoptar `os.path.join()` desde el principio, te preparas para escribir código más limpio, profesional y portable.

4. Un Paso Más Allá: Datos Estructurados con el Módulo `csv`

Los archivos `.txt` son geniales, pero ¿qué pasa si queremos guardar datos tabulares, como en una hoja de cálculo?

`producto,precio,stock"Teclado",85000,15"Mouse",45000,30`

Este formato se llama **CSV (Comma-Separated Values)**. Podríamos intentar leerlo línea por línea y usar `.split(',')`, pero es una mala idea. ¿Y si un producto tiene una coma en su nombre, como `"Monitor, 24 pulgadas"`? Nuestro código se rompería.

Python nos soluciona esto con el módulo `csv`, diseñado para manejar estos casos de forma robusta.

A. Leer un archivo `.csv`

```
import csv

# Creamos un archivo de ejemplo llamado 'inventario.csv' con el
contenido de arriba

total_valor_stock = 0
try:
    with open('inventario.csv', 'r') as archivo_csv:
        # csv.reader() nos devuelve un objeto especial para iterar de
forma segura
        lector_csv = csv.reader(archivo_csv)

        next(lector_csv) # Un truco útil para saltar la primera fila (el
encabezado)

        for fila in lector_csv:
            # Cada 'fila' es ahora una lista de strings: ['Teclado',
'85000', '15']
            nombre = fila[0]
            precio = float(fila[1]) # ¡No olvidar convertir los tipos de
datos!
```

```
            stock = int(fila[2])

            print(f"Procesando {nombre} con stock {stock}...")
            total_valor_stock += precio * stock

    print(f"\nEl valor total del inventario es:
${total_valor_stock:.2f}")

except FileNotFoundError:
    print("Error: No se encontró el archivo 'inventario.csv'.")
```

---

5. El Vistazo al Futuro: El Ciclo de Vida de los Datos

Con lo que aprendimos hoy, ya podemos implementar un programa completo que lee datos, los procesa usando todo nuestro conocimiento de funciones, listas y diccionarios, y genera un resultado persistente. Este es el núcleo de casi cualquier aplicación de software que maneja datos y será la base fundamental para el **Trabajo Práctico** que presentaremos la próxima clase.

---

6. Propuesta de Ejercicios Prácticos

Bloque 1: Fundamentos con `.txt`

1. **Bitácora Personal:** Escribe un programa que pregunte al usuario "¿Cómo estuvo tu día?". El programa debe tomar la respuesta, agregarle la fecha y hora actual al principio, y añadir esa entrada a un archivo llamado `bitacora.txt`. Cada vez que se ejecuta, agrega una nueva entrada sin borrar las anteriores.
2. **Lector de Artículos:** Guarda un párrafo de texto (de 3 o 4 líneas) en un archivo `articulo.txt`. Escribe un programa que lea el archivo y lo muestre en pantalla, pero numerando cada línea. Ej: "1. Lorem ipsum dolor...".

Bloque 2: Procesamiento con `csv`

3. **Calculadora de Promedio:** Crea un archivo `notas.csv` con dos columnas: `nombre_alumno` y `nota`. Agrega 5 filas con datos de ejemplo. Escribe un programa que lea el archivo, calcule el promedio de todas las notas e imprima un mensaje como "El promedio general del curso es: X.XX".

Bloque 3: Desafío Integrador

4. **Filtro de Productos:** Usando el `inventario.csv` del ejemplo de la clase, crea un programa que lea el archivo y genere un nuevo archivo llamado

`productos_a_reponer.txt`. Este archivo de salida solo debe contener los nombres de los productos cuyo stock sea menor a 20 unidades, uno por línea.

5. **Organizador de Reportes:** Escribe un programa que cree un archivo de texto simple llamado **`reporte.txt`** con el contenido "Este es un reporte automático.". Sin embargo, el archivo no debe crearse en el directorio actual, sino dentro de una subcarpeta llamada **`documentos`**. El script debe funcionar correctamente sin importar el sistema operativo.
    - **Pista:** Deberás usar `os.path.join()` para construir la ruta final.
6. **Copia de Seguridad Selectiva:** Pídele al usuario la ruta a un archivo de texto existente (ej: el **`articulo.txt`** del ejercicio 2). El programa debe leer todo el contenido de ese archivo y escribir una copia exacta en un nuevo archivo llamado **`articulo_copia.txt`**.

Bloque 4: Manejo Avanzado de Rutas y Archivos

7. **Generador de Logs Diarios:** Crea un programa que genere un archivo de log. El archivo debe guardarse dentro de una carpeta llamada **`logs`** (el script debe crear esta carpeta si no existe). El nombre del archivo debe ser la fecha actual, por ejemplo: **`2025-09-25.log`**. El contenido del archivo debe ser una sola línea: "Log del día: [fecha actual]".
    - **Pista:** Necesitarás el módulo `datetime` para obtener la fecha actual y `os.makedirs()` con el argumento `exist_ok=True`.
8. **Configurador Seguro:** Escribe un script que verifique si existe un archivo llamado **`config.ini`**.
    - Si el archivo **existe**, el programa debe imprimir: "Archivo de configuración encontrado. Cargando app...".
    - Si el archivo **no existe**, el programa debe crearlo, escribir en él la línea `modo=produccion` y luego imprimir: "No se encontró config.ini. Se ha creado uno por defecto.".
9. **Exportador CSV a Subcarpeta:** Toma el código del ejercicio 3 ("Calculadora de Promedio"). Modifícalo para que, además de imprimir el promedio en la consola, guarde el resultado en un archivo de texto. El archivo debe llamarse **`resumen_notas.txt`** y debe guardarse en una subcarpeta llamada **`resultados`**.
10. **Archivador de Proyectos:** Crea un programa que le pida al usuario el nombre de un nuevo proyecto. El script debe generar una estructura de carpetas estándar para ese proyecto:
    - Una carpeta principal con el nombre ingresado (ej: **`Proyecto_Final`**).
    - Dentro de esa carpeta, dos subcarpetas: **`data`** y **`reportes`**.
    - Finalmente, debe crear un archivo vacío llamado **`README.md`** dentro de la carpeta principal del proyecto.

11. **Consolidador de Datos (Desafío Mayor):** Crea dos archivos CSV, **`ventas_enero.csv`** y **`ventas_febrero.csv`**, y guárdalos dentro de una carpeta llamada **`ventas_historicas`**. Cada archivo debe tener las columnas `producto` y `monto`. Escribe un programa que lea ambos archivos, sume los montos totales de venta de los dos meses y escriba el resultado final en un archivo **`consolidado_anual.txt`** en la carpeta principal.